

1. Introduction and System Architecture

This guide provides the necessary technical information to maintain, deploy, and update the Pre-Pages Generator web application. The system architecture relies on three primary cloud platforms working together:

1. **GitHub:** Hosts the Python codebase and version control.
2. **Streamlit Community Cloud:** Hosts the live web application frontend and backend logic.
3. **Supabase:** Serves as the Backend-as-a-Service (BaaS) for persistent PostgreSQL database storage.

Important Security Note regarding Admin Accounts: For security reasons, administrative passwords and API keys are not included in this document. Please refer to the securely provided `credentials.txt` file (or contact the MMU Press) for the login credentials associated with the GitHub, Streamlit, and Supabase accounts.

2. GitHub Repository Management

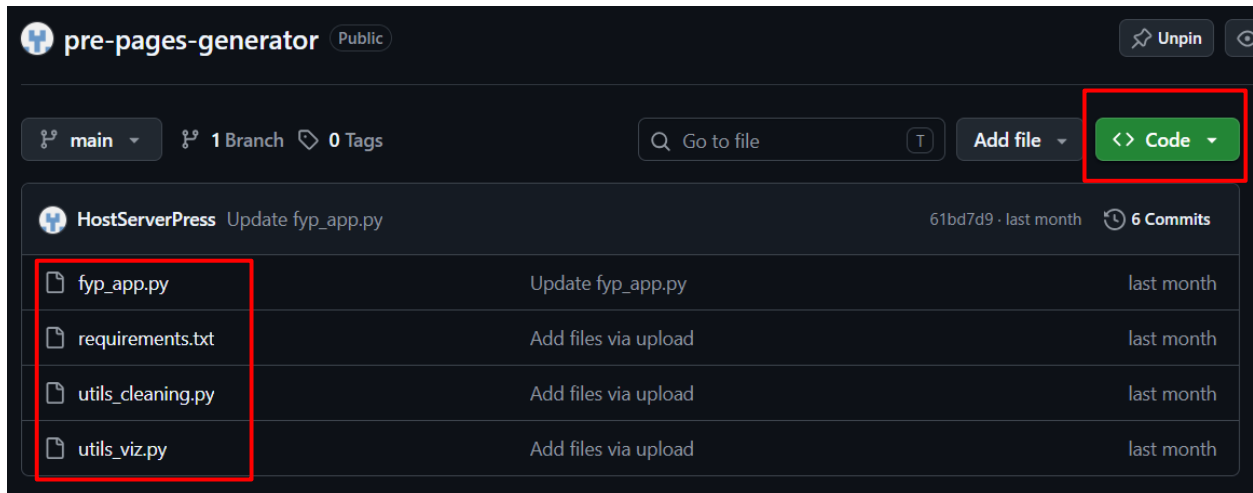
All source code for the application is stored in a dedicated GitHub repository. Streamlit Cloud continuously monitors this repository and automatically deploys the latest version whenever changes are pushed to the default **main** branch.

Key Files in the Repository:

- `fyp_app.py`: The main Python script containing the core Streamlit UI components, navigation, and wizard logic.
- `utils_cleaning.py`: A modular utility file that handles background data extraction, dataframe manipulation, and text sanitization.
- `utils_viz.py`: A modular utility file containing the logic for the Data Visualiser tab (such as generating the analytical charts and the Topic Trends Word Cloud).
- `requirements.txt`: Lists all required Python libraries (e.g., `streamlit`, `pandas`, `supabase`, `docxtempl`, `beautifulsoup4`). If a new library is added to the project, it must be declared here.

(Note: There is no template folder in the repository. The system is designed to be fully dynamic, meaning Word templates are uploaded manually by the user during Step 1 in the web app.)

How to Update the Live App:



To update the live web application, a developer simply needs to push or upload their updated Python code to the `main` branch of this GitHub repository. Streamlit will detect the commit and automatically begin a rebuild.

3. Streamlit Community Cloud (Hosting & Deployment)

Streamlit Community Cloud is where the application is hosted and executed.

Waking a Sleeping Application



Zzzz

This app has gone to sleep due to inactivity. Would you like to wake it back up?

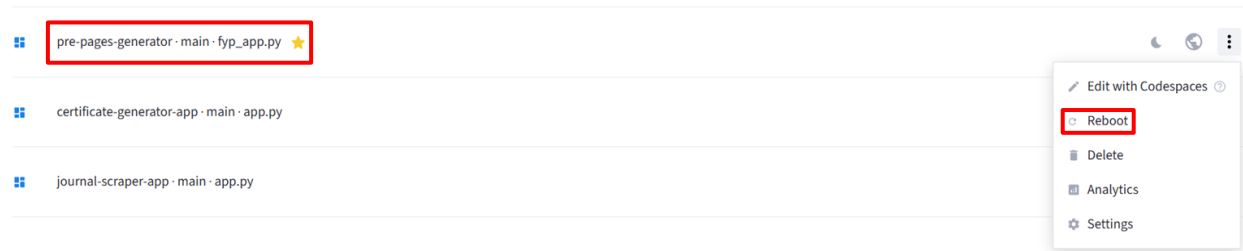
[Yes, get this app back up!](#)

If you believe this is a bug, please [contact us](#) or [visit the Streamlit forums](#).

To conserve server resources, Streamlit automatically puts the application to sleep after a period of inactivity. When anyone navigates to the application URL during this state, they will see a screen stating, *"This app has gone to sleep due to inactivity. Would you like to wake it back up?"* The user simply needs to click the **Yes, get this app back up!** button. The server will spin up automatically within a few moments without requiring administrative dashboard access.

Rebooting the Application (Clearing Server Load)

hostserverpress' apps



If the application crashes, encounters a memory error, or slows down significantly due to heavy usage over time, an administrator can manually reboot it. Rebooting clears the server's RAM and resets the load. To do this, click the three dots next to the application name (or in the top right corner of the live app) and select **Reboot app** to restart the server instance.

Environment Variables (Secrets Management)

App settings

General

Sharing

Secrets

Secrets

Provide environment variables and other secrets to your app using [TOML](#) format. This information is encrypted and served securely to your app at runtime. Learn more about Secrets in our [docs](#). Changes take around a minute to propagate.

```
SUPABASE_URL = "[REDACTED]"
SUPABASE_KEY = "[REDACTED]"
```

Save changes

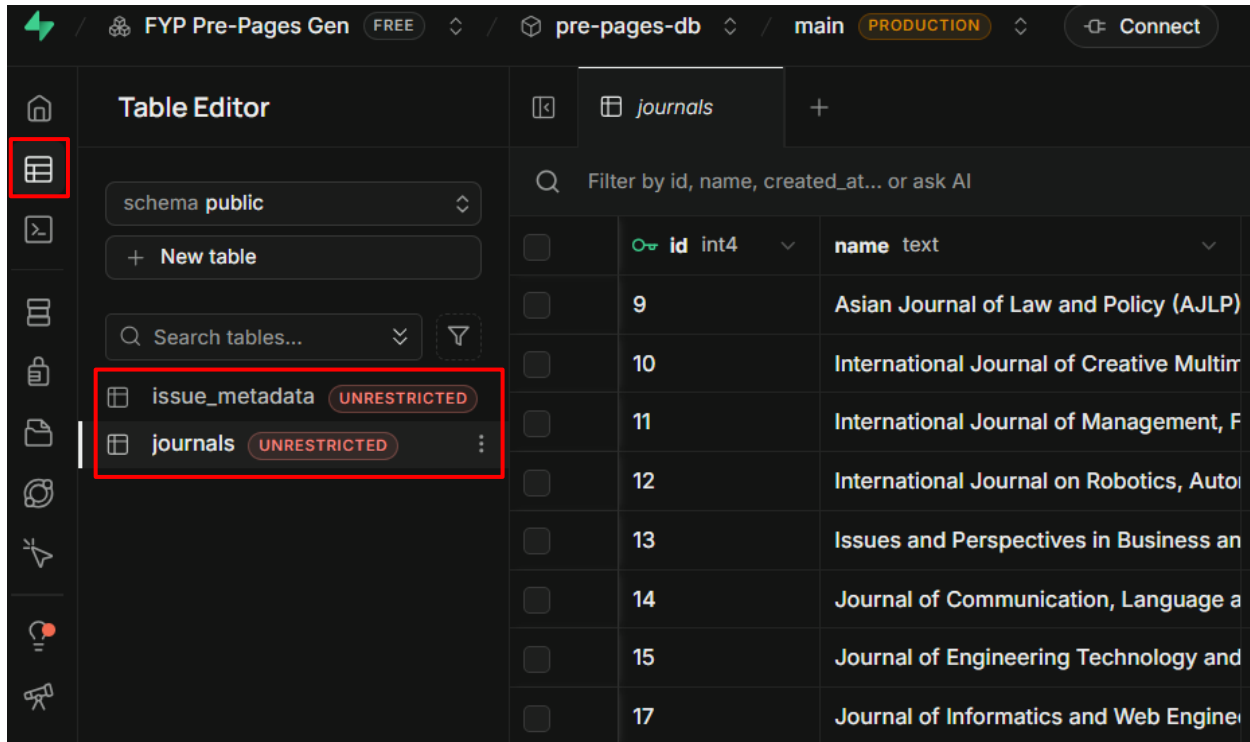
The application relies on private API keys (such as the Supabase URL and Supabase Key) to function. These must never be hardcoded into the GitHub repository. Instead, they are stored securely in Streamlit's **Secrets** manager. If the database is ever migrated, the new keys must be updated here:

1. Go to App Settings > Secrets.
2. Update the variables (e.g., `SUPABASE_URL="your_url"` and `SUPABASE_KEY="your_key"`).
3. Save the settings. The app will automatically reboot to apply the new keys.

4. Supabase (Database Management)

Supabase provides the PostgreSQL database used to store persistent journal metadata (Journal Name, eISSN, Publisher Address).

Accessing the Database Schema



While the frontend application allows editors to add or delete journals via the UI, developers can manage the raw data directly through the Supabase **Table Editor**.

1. Log into the Supabase project.
2. Navigate to the **Table Editor** on the left menu.
3. Select the **journals** table. Here, developers can manually insert, update, or delete rows, or modify the table schema (adding new columns if future updates require more metadata).

Row Level Security (RLS)

If API access errors occur when the Streamlit app attempts to read or write to the database, ensure that the Supabase Row Level Security (RLS) policies are configured correctly to allow the Streamlit service role to perform **SELECT**, **INSERT**, and **DELETE** operations.

5. Local Development Setup

If a developer needs to test new features or troubleshoot bugs safely without affecting the live production application, they should set up a local development environment.

Step-by-Step Local Setup:

1. Clone the repository to the local machine: `git clone <repository_url>`
2. Create a virtual environment and activate it: `python -m venv venv`
3. Install the necessary dependencies: `pip install -r requirements.txt`
4. Create a `.streamlit/secrets.toml` file in the root directory and add the necessary Supabase API keys to simulate the cloud environment.
5. Run the application locally: `streamlit run app.py`

This will launch a local instance of the application in the developer's web browser, allowing them to test code changes safely before pushing the final version to GitHub.